CONCLAVE

A Practical Guide



Windows & VS Code • Using the framework • The models • Raspberry Pi • The mathematics

What this document answers

1. How do I get this running on Windows in VS Code?
2. How do I actually use the framework in my own code?
3. Which machine-learning models are inside, and how do I change them?
4. How do I deploy and test it for a Raspberry Pi 4 without owning one?
5. Where does every equation in the paper come from, and what does it mean?

CONCLAVE is an agentic-AI framework whose cognitive core is a heterogeneous machine-learning ensemble rather than a language model. It runs offline, needs no GPU and no API key, and the disaster agent serialises to 232 KB.

90.8% tool routing | **4.0 ms** per decision | **232 KB** whole agent | **0** network calls

Aranya Mahmud

Department of Computer Science and Engineering
Daffodil International University, Dhaka, Bangladesh

Companion to *CONCLAVE: Agentic AI Without a Language Model*
Framework version 0.1.0 • Apache-2.0

Contents

1 Orientation: what you are looking at **3**
- 1.1 The seven objects you will actually touch . . . 3
- 1.2 If you have twenty minutes . . . 3

2 Running it on Windows with VS Code **4**
- 2.1 Prerequisites . . . 4
- 2.2 One-time setup . . . 4
  - 2.2.1 What the three extras install . . . 4
- 2.3 Point VS Code at the interpreter . . . 5
- 2.4 Check it works . . . 5
- 2.5 The debugger, which is where the real understanding happens . . . 6
- 2.6 Reproducing the paper . . . 6
- 2.7 When something breaks . . . 7

3 Using the framework **7**
- 3.1 Step 1 — declare what the agent can do . . . 7
- 3.2 Step 2 — observations . . . 8
- 3.3 Step 3 — featurise under a budget . . . 9
- 3.4 Step 4 — the brain . . . 9
- 3.5 Step 5 — the gate and the safety layer . . . 9
- 3.6 Step 6 — assemble and decide . . . 10
- 3.7 What a `Decision` carries . . . 11
- 3.8 Running a sequential environment . . . 11
- 3.9 Tool routing for open vocabularies . . . 11

4 The machine-learning models inside **12**
- 4.1 What is actually in the committee . . . 12
- 4.2 Two defaults that came from measurement, not taste . . . 12
  - 4.2.1 `n_jobs=1` everywhere . . . 12
  - 4.2.2 Shallow, few trees . . . 13
- 4.3 Swapping the models . . . 13
- 4.4 Choosing a feature budget . . . 14
- 4.5 Training on your own data . . . 14
- 4.6 Where classical models genuinely lose . . . 15

5 Deploying and testing for a Raspberry Pi 4 **15**
- 5.1 Three levels of fidelity . . . 16
- 5.2 The finding that will save you a weekend . . . 16
- 5.3 Setting up emulated ARM64 on Windows . . . 17
- 5.4 Build and run the Pi target . . . 17
- 5.5 The compose profiles . . . 17
- 5.6 What `pi_check.py` reports, and why it is split . . . 18
- 5.7 What you may and may not claim . . . 19
- 5.8 When you do get real hardware . . . 19
- 5.9 Deploying a trained agent, not the training code . . . 20

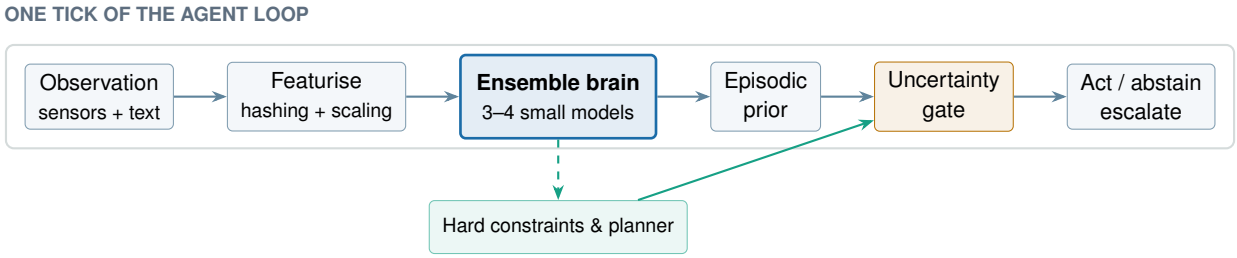
6 The mathematics: where every equation comes from **20**
- 6.1 Entropy, and why it is normalised . . . 21
- 6.2 Aggregating the committee . . . 21
- 6.3 The decomposition — the one equation in the paper . . . 21
  - 6.3.1 Where it comes from . . . 21

6.3.2 Why the epistemic term cannot go negative . . . 22
6.3.3 What the two terms actually mean . . . 22
6.4 Margin and vote entropy . . . 22
6.5 Selective prediction: the risk–coverage curve . . . 23
6.6 Text features: TF–IDF and the hashing trick . . . 23
6.6.1 TF–IDF, exactly as scikit-learn computes it . . . 23
6.6.2 The hashing trick, and what it costs . . . 23
6.7 Episodic memory as a kernel-weighted prior . . . 24
6.8 The planner, and an honest caveat . . . 24
6.9 The paired bootstrap in the headline experiment . . . 25
6.10 Reading list, in dependency order . . . 25

Appendix — one-page reference **25**

1 Orientation: what you are looking at

Before the instructions, a map. CONCLAVE replaces one component of a conventional AI agent — the language model at the centre — with a committee of small machine-learning models. Everything else an agent needs is still there.



1.1 The seven objects you will actually touch

Object	What it is	Lives in
`Observation`	One reading of the world: numeric features plus optional text	`conclave/core/types.py`
`FusedFeaturizer`	Turns observations into vectors under a fixed memory budget	`conclave/brain/featurize.py`
`EnsembleBrain`	The committee. Replaces the LLM	`conclave/brain/ensemble.py`
`UncertaintyGate`	Converts disagreement into act / gather / abstain / escalate	`conclave/brain/uncertainty.py`
`ToolRegistry`	Your capabilities, with typed schemas	`conclave/core/registry.py`
`Agent`	The loop that ties it together	`conclave/core/agent.py`
`ToolRouter`	Picks a tool by ranking, not classifying	`conclave/brain/router.py`

The one idea worth carrying through this document

A single model gives you a prediction and a confidence number. A *committee* gives you something a single model cannot: the **spread of opinion** between members. CONCLAVE treats that spread as a first-class control signal — it is the difference between "this is a hard call" and "I have no idea", and those two situations warrant different behaviour. Section 6 derives it properly.

1.2 If you have twenty minutes

1. §2 — set up the virtual environment, run `pytest`. Expect **14 passed**.
2. Run `python demo.py episode` and watch the `net` column. Long stretches say `DOWN` and the agent keeps working. That is the entire thesis of the project, visible in one column.
3. Run `python demo.py uncertainty` to see the committee split apart as you break its sensors.

Everything else in this guide is depth on those three things.

2 Running it on Windows with VS Code

Nothing here is exotic. If you have set up a virtual environment for a Flask project you already know this workflow; the only unusual part is that a couple of the experiments spend a few minutes on CPU.

2.1 Prerequisites

Thing	Version	Why
Python	**3.10 – 3.12**	3.13 still misses `lightgbm` wheels on some Windows builds
VS Code	any current	
Python extension	`ms-python.python`	brings `debugpy` and the Testing panel
Disk	≈ 700 MB	mostly `scikit-learn` and `lightgbm` wheels
Network	first install only	the framework never touches the network at runtime

There is deliberately no CUDA, no GPU, and no API key anywhere in this list.

2.2 One-time setup

Open the `conclave` folder in VS Code — **File → Open Folder**, and pick the folder that *contains* `pyproject.toml`, not its parent. This matters: pointing VS Code at the parent is the single most common reason `pytest` later collects zero tests.

Then open the integrated terminal with Ctrl + `:

PowerShell — one time

```
py -3.12 -m venv .venv
.\.venv\Scripts\Activate.ps1

python -m pip install --upgrade pip
pip install -e ".[boost,experiments,dev]"
```

Your prompt should now start with `(.venv)`.

If PowerShell refuses to run the activation script

Windows blocks unsigned scripts by default. Unblock it for this terminal only:

```
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
```

`-Scope Process` means the change dies with the terminal window, which is exactly what you want — do not use `-Scope CurrentUser` for this.

2.2.1 What the three extras install

`pip install -e ".[boost,experiments,dev]"` pulls three optional groups on top of the core dependencies:

Extra	Packages	Needed for
(core)	numpy, scipy, scikit-learn, joblib, psutil	the framework itself
`boost`	lightgbm, xgboost	the gradient-boosted committee member
`experiments`	pandas, pyarrow, matplotlib	running `experiments/` and drawing figures
`dev`	pytest	the test suite

LightGBM is optional by design

If `lightgbm` will not install, the framework does not break. `ToolRouter` falls back to scikit-learn's `HistGradientBoostingClassifier`, which costs about half a point of routing accuracy. You can verify which one you got:

```
from conclave.brain.router import ToolRouter
print(list(ToolRouter()._default_members()))    # ['lgbm', ...] or ['hgb', ...]
```

2.3 Point VS Code at the interpreter

Press Ctrl+Shift+P → **Python: Select Interpreter** → choose the one under `.venv`.

The repository ships a `.vscode/settings.json` that already sets this, enables pytest, and hides `__pycache__` from the explorer, so in most cases it happens by itself:

.vscode/settings.json — the parts that matter

```
"python.defaultInterpreterPath": "${workspaceFolder}/.venv/Scripts/python.exe",
"python.testing.pytestEnabled": true,
"python.testing.pytestArgs": ["tests"],
"python.analysis.extraPaths": ["${workspaceFolder}"]
```

2.4 Check it works

PowerShell

```
pytest -q
```

Expected

14 passed in roughly six seconds. Four `ConvergenceWarning`s from `SGDClassifier` are normal — that member is deliberately capped at 25 iterations because it is a committee member, not a soloist.

The 14 tests are two distinct groups, and the distinction is worth understanding:

File	What it protects
`tests/test_smoke.py`	The framework: uncertainty maths, planner, tool registry, memory, a full agent episode, constraint escalation. Nine tests.
`tests/test_paper_claims.py`	Every number quoted in the paper, checked against `results/*.json`. Five tests.

Why the second group exists

`test_paper_claims.py` fails *on purpose* if you re-run an experiment and a number moves. That is not a broken test — it is the mechanism that stops the paper and the code drifting apart. When it fails, look at the diff and decide which one is wrong before you touch either.

VS Code's Testing panel (the conical-flask icon in the sidebar) picks these up automatically and lets you run them one at a time.

2.5 The debugger, which is where the real understanding happens

The repository ships eight launch configurations in `.vscode/launch.json`. Press F5 and pick one from the dropdown. The most useful by a wide margin:

Configuration	What it does
Demo: watch an episode	The agent decides tick by tick with the network failing
Demo: tool routing (interactive)	Type a request, watch it score every tool
Exp 3: connectivity loss	The headline experiment, about three minutes
Debug: break inside Agent.decide	Same as the episode demo but with `justMyCode: false`, so you can step into scikit-learn

The single most useful breakpoint in the project

Open `conclave/core/agent.py`, find `Agent.decide`, and put a breakpoint on the line

```
outcome = self.gate(rep.stats)
```

Launch **Debug: break inside Agent.decide**. When it stops, inspect `rep` in the Variables pane. You get `rep.member_probs` — what each committee member believed — and `rep.stats`, the full uncertainty decomposition for that one decision. Everything in §6 is visible there as concrete numbers.

2.6 Reproducing the paper

PowerShell — times are for a modest laptop CPU

```
python experiments/exp3_sequential.py     # ~3 min, no data needed
python experiments/exp5_uncertainty.py    # ~4 min, no data needed
python experiments/exp4_edge.py           # ~1 min, no data needed
python experiments/exp1_tool_routing.py   # ~2 min, needs data/bfcl
python experiments/exp2_humaid.py 12 14   # ~4 min, needs data/humaid
python experiments/make_figures.py        # redraws paper/figures/

python scripts/run_all.py                 # or all of the above
```

`data/` is already in the repository (19 MB), so every one of these works offline right now. To re-fetch it: `python scripts/fetch_data.py`.

Each experiment writes a JSON artefact to `results/` recording platform, seed and git commit, so any number can be traced back to the run that produced it.

One genuine Windows difference

`exp4_edge.py` enforces a RAM ceiling using `RLIMIT_AS`, which is POSIX-only. On Windows the code detects this, prints a notice, and reports thread caps alone — the memory ceiling becomes advisory rather than enforced.
Everything else is identical across platforms. But do not report an *enforced* memory ceiling in a paper table from a Windows run, because on Windows it was not enforced. §5 shows how to get a genuinely enforced ceiling using containers.

2.7 When something breaks

Symptom	Cause and fix
`ModuleNotFoundError: conclave`	The venv is not active, or you skipped `pip install -e .`. Check for `(.venv)` in the prompt.
`lightgbm` fails to install	You are on Python 3.13. Use 3.12, or accept the automatic `HistGradientBoosting` fallback.
`pytest` collects 0 tests	VS Code has the parent folder open. Reopen the `conclave` folder itself.
Activation script blocked	`Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass`
Experiments feel slow	Expected. `exp3` runs 3,600 episodes and trains a committee for each configuration.
`test_paper_claims` fails	You re-ran an experiment and a number moved. The test is doing its job — inspect the diff.

3 Using the framework

This section builds a complete agent from nothing. The running example is a solar-powered borehole pump serving a village: it has a tank-level sensor, a turbidity probe, a battery monitor and a flow meter, plus a text field where a caretaker types what they see. There is no reliable network. The full file is `examples/pump_station.py` — run it with `python examples/pump_station.py`.

The shape of every CONCLAVE program

tools → training observations → featuriser → brain → gate + constraints → agent.

Six steps, in that order, always. The rest is detail.

3.1 Step 1 — declare what the agent can do

A tool is a Python function plus a schema. The `@tool` decorator infers the schema from the type annotations, the way a typed route converter does:

Tools

```
from conclave.tools.base import tool
from conclave import ToolRegistry

@tool(description="Start the borehole pump to refill the tank")
def start_pump(minutes: int = 30) -> str:
```

```
    return f"pump running {minutes} min"

@tool(description="Open the chlorine doser to treat the tank", irreversible=True)
def dose_chlorine(ml: int = 50) -> str:
    return f"dosed {ml} ml"

@tool(description="Send an SMS to the maintenance technician")
def call_technician(reason: str = "unknown") -> str:
    return f"SMS sent: {reason}"

registry = ToolRegistry([start_pump, dose_chlorine, call_technician])
```

Three arguments to `@tool` carry real weight:

Argument	Effect
`description`	Becomes matchable text for `ToolRouter`. Write it the way a user would describe the need, not the way you named the function.
`irreversible`	Marks the tool for the constraint layer and lets the planner exclude it. Dosing chemicals is irreversible; sending an SMS is not.
`preconditions` / `effects`	Predicate lists the planner searches over. Leave empty if you are not planning.

3.2 Step 2 — observations

An `Observation` carries numeric features, optional free text, and metadata. Either modality may be empty; the featuriser handles whichever is present.

Observations

```
from conclave import Observation

Observation(
    features={"tank_level_pct": 11.0, "turbidity_ntu": 1.2,
              "battery_volts": 12.9, "flow_lpm": 0.0},
    text="tank nearly empty this morning",
)
```

Training data is a list of observations and a matching list of action names. In the example these are generated synthetically, five scenarios × 120 samples; in a real deployment they are logged decisions from whoever runs the station today.

Two constraints on your training data

- `EnsembleBrain` needs **at least two classes**, and enough examples per class for its internal cross-validation. A handful of rows will raise.
- Class names become action names. They must match tool names exactly, or `Agent` will decide on an action that is not in the registry and silently do nothing.

3.3 Step 3 — featurise under a budget

Featurisation

```
from conclave.brain.featurize import (BudgetedTextFeaturizer, FusedFeaturizer,
                                      TabularFeaturizer)

fz = FusedFeaturizer(
    text=BudgetedTextFeaturizer(n_word_features=2**12, n_char_features=2**12),
    tabular=TabularFeaturizer(),
).fit(observations)

X = fz.transform(observations)
```

The feature budget is the deployment knob

`BudgetedTextFeaturizer` uses the hashing trick, so it stores **no vocabulary at all** — its size is fixed when you construct it and does not grow with your corpus. `n_word_features` and `n_char_features` therefore set the model size directly: 2^{12} gives a 2.6 MB artefact, 2^{16} gives 36 MB. Table III of the paper shows what that costs in accuracy. §6 explains why the hashing trick works and what it costs statistically.

3.4 Step 4 — the brain

The committee

```
from conclave import EnsembleBrain

brain = EnsembleBrain().fit(X, labels)

print(list(brain.members_))                          # ['logreg', 'sgd_mh', 'nb']
print(brain.weights_.round(3))                       # [0.333 0.333 0.333]
print(brain.footprint_bytes()["_total"] / 1000)      # 24.2   (KB)
```

Twenty-four kilobytes. That is the entire decision-making apparatus for this agent. §4 covers what is inside and how to change it.

3.5 Step 5 — the gate and the safety layer

Uncertainty gate and hard constraints

```
from conclave import UncertaintyGate
from conclave.planning.rules import ConstraintSet, require_confidence

gate = UncertaintyGate(epistemic_max=0.30,    # committee disagreement ceiling
                       aleatoric_max=0.75,    # genuine-ambiguity ceiling
                       confidence_min=0.45,
                       margin_min=0.05,
                       allow_gather=False)    # no cheap extra sensing here

# No statistical model may dose chemicals on a weak majority.
constraints = ConstraintSet([require_confidence(0.80, ["dose_chlorine"])])
```

The constraint layer sits *above* the learned policy

A violated constraint does not adjust a probability — it converts the action into a recorded `ESCALATE`. This is deliberate: no statistical model should be able to authorise an irreversible act on its own, however confident it claims to be.

Rather than hand-tuning those four thresholds, you can calibrate them against a target precision on held-out data:

Calibrating instead of guessing

```
gate = brain.calibrate_gate(X_val, y_val, target_precision=0.90)
print(gate.calibration_)
# {'target_precision': 0.9,
#  'achieved': {'coverage': 0.52, 'precision': 0.906}, ...}
```

Read that as: *acting on 52% of decisions and deferring the rest buys 90.6% accuracy on the ones acted upon.* §6.5 explains the selective-prediction machinery behind it.

3.6 Step 6 — assemble and decide

The agent

```
from conclave import Agent, EpisodicMemory

agent = Agent(brain=brain, featurizer=fz, registry=registry, gate=gate,
              constraints=constraints, memory=EpisodicMemory(capacity=500))

d = agent.decide(Observation(features=..., text="very cloudy water"))
print(d.action, d.confidence, d.disagreement, d.gate)
```

Running the four probe observations from the example file gives:

Actual output of examples/pump_station.py

```
observation                          action             conf  disag      gate
tank nearly empty this morning       <escalate>         0.65   0.40  escalate
very cloudy water since the storm    dose_chlorine()    1.00   0.00       act
no flow and the battery looks flat   call_technician()  0.78   0.19       act
strange smell from the wellhead ...  <escalate>         0.46   0.35  escalate
```

All four rows are worth reading carefully, because two of them are the agent working and two are the agent *knowing it should not act*:

Row	What actually happened
cloudy water	Textbook case. Every member agrees (disagreement 0.00), confidence 1.00 clears the 0.80 chlorine constraint, the tool fires.
no flow, flat battery	Correct call at 0.78 confidence with mild disagreement. Note it would *not* have been allowed to dose chlorine at this confidence.
strange smell	Nothing like it in training. The committee splits (0.35), the gate escalates rather than guessing. **This is the behaviour the whole design exists to produce.**
tank nearly empty	An honest failure. Training said "tank low overnight draw down"; the probe says "nearly empty". Hashed n-grams have no idea those mean the same thing, so the members split and it escalates.

That last row is a real limitation, not a bug

CONCLAVE's text features are lexical. They have no compositional semantics, so paraphrase, negation and synonymy hurt it more than they hurt a transformer. The paper states this in its limitations; the example reproduces it rather than hiding it. If your text varies a lot in

phrasing, either widen your training vocabulary or lean harder on the numeric features, which have no such problem.

3.7 What a `Decision` carries

Every decision is fully auditable — this matters when an agent is making consequential calls unsupervised:

Field	Meaning
`action`	What it decided: a tool call, or abstain / gather / escalate
`posterior`	Probability over every action
`member_votes`	What *each* committee member believed, separately
`confidence`	Largest posterior probability
`disagreement`	Normalised epistemic uncertainty — the spread of opinion
`entropy`	Normalised total uncertainty
`margin`	Top-1 minus top-2 probability
`gate`	Which branch fired, and therefore why
`latency_ms`	Wall-clock cost of this decision

3.8 Running a sequential environment

`agent.decide()` handles one observation. For a control loop, `agent.run()` drives any environment exposing `reset()` and `step(action)`:

Episodes

```
from conclave.envs.disaster import DisasterConfig, FloodResponseEnv

episode = agent.run(FloodResponseEnv(DisasterConfig(outage_prob=0.3)),
                    max_steps=40)

print(episode.total_reward, episode.n_escalations)
print(episode.to_dict())          # JSON-serialisable audit trail
```

With `memory` attached and `learn_online=True` (the default), successful steps are written back into episodic memory as cases, so the agent adapts within a deployment without retraining.

3.9 Tool routing for open vocabularies

`EnsembleBrain` classifies into a *fixed* action set. When tools come and go at runtime, use `ToolRouter` instead: it scores *(request, tool)* pairs, so a tool registered after training is scored by the same features as any other.

Routing

```
from conclave.brain.router import ToolRouter

router = ToolRouter(abstain=True, abstain_threshold=0.35).fit(
    queries, candidate_sets, targets)

result = router.route("people stuck on a roof in sector 4, send help",
                      registry.specs())
```

```
print(result.tool, result.scores, result.latency_ms)
```

`result.tool` is `None` when nothing scores above the abstention threshold — which is the correct answer more often than agent frameworks like to admit. Try `python demo.py route` and give it something outside its competence; it will show you flat scores rather than a confident mistake.

4 The machine-learning models inside

This is the part that replaces the language model, so it deserves scrutiny. The short version: CONCLAVE runs a small committee of *deliberately different* scikit-learn estimators and averages their posteriors. The diversity is not incidental — it is the whole mechanism.

4.1 What is actually in the committee

`EnsembleBrain` picks its default members based on your data. There are three cases, all decided automatically in `conclave/brain/ensemble.py`:

Your data	Members	Why these
Sparse, non-negative (text only)	`LogisticRegression` `SGDClassifier` `ComplementNB`	Discriminative linear, a differently-regularised linear, and a *generative* model. Complement NB is the strongest naive-Bayes variant for imbalanced text.
Sparse with negatives (text + scaled sensors)	`LogisticRegression` `SGDClassifier` `BernoulliNB`	Complement NB rejects negative inputs, which appear the moment you standardise tabular features. Bernoulli NB binarises at zero and accepts either, so the committee keeps a generative member instead of silently degrading to two linear models.
Dense (sensors only)	`LogisticRegression` `RandomForest` `ExtraTrees`	Linear plus two differently-randomised tree ensembles. Trees dominate on low-dimensional tabular data.

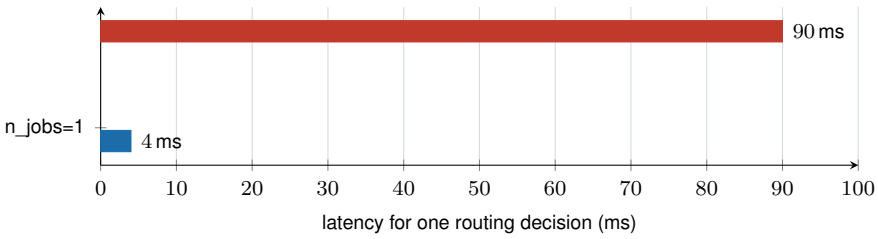
Why heterogeneous rather than one big random forest?

Because epistemic uncertainty is only meaningful if members can disagree for *structural* reasons. A random forest's trees share an inductive bias, so off-distribution they tend to be confidently wrong *together* — their disagreement collapses exactly when you need the warning most. Mixing model families keeps the disagreement signal alive where it matters.

4.2 Two defaults that came from measurement, not taste

4.2.1 `n_jobs=1` everywhere

This looks like a mistake and is not. An agent decides about *one* observation at a time, over a handful of candidate actions. At that batch size, constructing a joblib thread pool costs far more than the prediction:



Parallelism is a *throughput* optimisation and a *latency* pessimisation. Edge agents want the latter. If you are ever scoring in bulk — offline evaluation, say — pass `n_jobs=-1` explicitly for that call.

4.2.2 Shallow, few trees

Depth-10 forests with 60 estimators lose well under a point of accuracy against depth-unbounded 200-tree forests, and cut the serialised model roughly 9×. Measured on BFCL routing:

Committee	Top-1	ms/dec.	Size
RF200 + ET200, `n_jobs=-1`	88.6%	90.0	5 519 KB
RF200 + ET200, `n_jobs=1`	88.6%	12.5	5 519 KB
RF100 + ET100, depth 12	90.2%	7.4	1 332 KB
RF60 + ET60, depth 10	89.2%	5.2	635 KB
LGBM150 + LogReg	90.2%	2.2	222 KB
LGBM200 + RF60d10 + LogReg *(default)*	**90.8%**	**4.0**	**631 KB**

4.3 Swapping the models

Any scikit-learn-compatible classifier works, provided it exposes `predict_proba` or `decision_function` (the latter gets softmaxed automatically). Pass a dict:

A custom committee

```python
from sklearn.linear_model import LogisticRegression, RidgeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.neighbors import KNeighborsClassifier
from conclave import EnsembleBrain

brain = EnsembleBrain(
    members={
        "logreg": LogisticRegression(C=2.0, max_iter=500, n_jobs=1),
        "forest": RandomForestClassifier(n_estimators=80, max_depth=10, n_jobs=1),
        "knn":    KNeighborsClassifier(n_neighbors=7),        # instance-based
        "ridge":  RidgeClassifier(alpha=1.0),                 # no predict_proba
    },
    weighting="logloss",      # "uniform" | "accuracy" | "logloss"
    cv=3,
).fit(X, y)

print(brain.weights_.round(3))
print(brain.fit_seconds_)
```

`weighting`	Behaviour
`"uniform"`	Plain average. The most robust default, and what the paper uses. Hard to beat when members are comparable.
`"accuracy"`	Weight by out-of-fold accuracy. Helps when one member is clearly weaker.
`"logloss"`	Weight by inverse out-of-fold log loss. Rewards *calibration*, not just correctness — usually the better choice when the posterior itself is being consumed downstream, as it is here by the gate.

Adding members has a cost you can measure

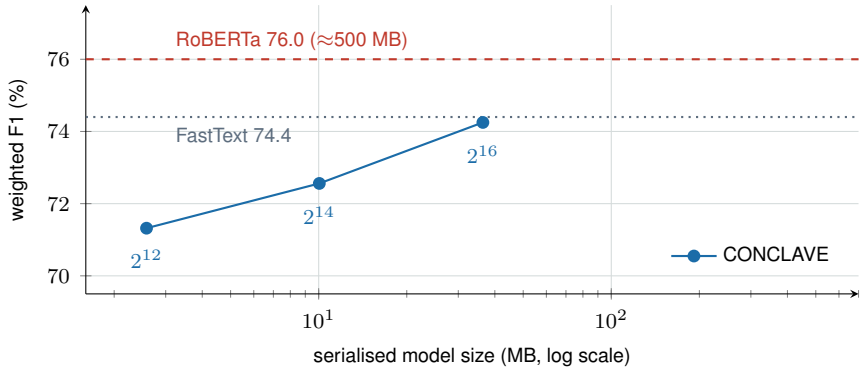
Every extra member adds its full inference time to *every* decision, and its serialised bytes to the deployment. Before adding one, check it earns its place:

```
print(brain.footprint_bytes())    # per-member size, and the total
print(brain.fit_seconds_)         # per-member training time
```

Three to four members is the sweet spot in every configuration we measured.

4.4 Choosing a feature budget

The budget is the single most consequential deployment decision, because the hashing trick makes model size a constant you choose rather than a function of your corpus. Measured on HumAID:



A practical rule for picking one:

Budget	Artefact	Use when
2^{12}	2.6 MB	Microcontroller-class hardware, or text is a minor signal
2^{14}	10 MB	Pi Zero / Pi 3 with other things running
2^{16}	36 MB	Pi 4 or better; the accuracy plateau starts around here

4.5 Training on your own data

The whole training path

```
import joblib
from conclave import EnsembleBrain
from conclave.brain.featurize import (BudgetedTextFeaturizer, FusedFeaturizer,
                                      TabularFeaturizer)
```

```
fz = FusedFeaturizer(
    text=BudgetedTextFeaturizer(n_word_features=2**14, n_char_features=2**14),
    tabular=TabularFeaturizer(),
).fit(train_obs)

brain = EnsembleBrain(weighting="logloss").fit(fz.transform(train_obs), train_y)
brain.calibrate_gate(fz.transform(val_obs), val_y, target_precision=0.90)

joblib.dump({"fz": fz, "brain": brain, "gate": brain.gate},
            "agent.joblib", compress=3)
```

Loading on the target device

```
import joblib
from conclave import Agent

bundle = joblib.load("agent.joblib")
agent = Agent(brain=bundle["brain"], featurizer=bundle["fz"],
              registry=registry, gate=bundle["gate"])
```

joblib files are executable code

`joblib.load` unpickles, and unpickling an untrusted file runs arbitrary code. Only load artefacts you produced yourself or received over a channel you trust. This is a property of pickle, not of CONCLAVE, but it is your problem the moment you ship models to field devices.

4.6 Where classical models genuinely lose

Being straight about this is more useful than a sales pitch:

Weakness	What to do about it
No compositional semantics	Paraphrase and negation break lexical features. Widen training vocabulary, or weight numeric features more heavily.
Fixed action set (`EnsembleBrain`)	Use `ToolRouter` when tools change at runtime — it ranks pairs, so unseen tools still score.
No argument extraction	CONCLAVE selects a tool; it does not fill typed arguments from free text. Use `arg_builder` on `Agent` for rule-based slots.
Needs labelled decisions	There is no zero-shot mode. You need examples of the decision being made, from logs or from an expert controller.

5 Deploying and testing for a Raspberry Pi 4

You do not own a Raspberry Pi. That is a normal situation and it does not stop you from doing most of this properly — provided you are precise about which claims survive emulation and which do not. Conflating those two is how resource claims get rejected in review.

5.1 Three levels of fidelity

Level	What it is	What it can prove
1. Constrained x86 what `exp4_edge.py` does	Thread caps and a RAM ceiling on your own machine	Memory footprint, algorithmic behaviour. Latency only as an *x86* number.
2. Emulated ARM64 Docker + QEMU	A genuine `aarch64` userland: ARM wheels, ARM BLAS, ARM NumPy paths	That it *installs*, *runs* and *fits* on ARM. Correctness. **Not** latency.
3. Real hardware	An actual Pi 4, about $50	Everything, including latency and energy

What emulation is genuinely for

Level 2 answers the question that actually blocks a deployment: *does the software stack exist for this architecture at all?* Whether `scipy` has an ARM wheel is a hard yes/no that emulation answers definitively, and it is a far more common cause of failed edge deployments than being 3 ms too slow.

5.2 The finding that will save you a weekend

Raspberry Pi OS ships in 32-bit (`armv7l`) and 64-bit (`aarch64`) flavours. For this dependency stack they are not equivalent:

Package	PyPI `aarch64` (64-bit)	PyPI `armv7l` (32-bit)
numpy	wheel	none
scipy	wheel	none
scikit-learn	wheel	none
lightgbm	wheel	none
psutil	wheel	none
joblib	pure Python — fine either way	

Use the 64-bit OS

On 32-bit Raspberry Pi OS, `pip install scipy` means *compiling scipy on the Pi.* That is hours of build time and it frequently dies of memory exhaustion on a 1 GB board.
Two ways out, in order of preference:

1. **Install 64-bit Raspberry Pi OS** and use ordinary PyPI wheels. The Pi 4 and Pi Zero 2 W are both 64-bit capable. This is the right answer.
2. If you are stuck on 32-bit, use **piwheels** (`piwheels.org`), which publishes pre-built `armv7l` wheels and is already configured as an index on Raspberry Pi OS. It carries builds for all four packages above.

5.3 Setting up emulated ARM64 on Windows

PowerShell — one time

```
# 1. Docker Desktop with the WSL2 backend must be running.
docker version

# 2. Register the QEMU binfmt handlers so the kernel can execute ARM binaries.
docker run --privileged --rm tonistiigi/binfmt --install arm64

# 3. Confirm arm64 is now a build target.
docker buildx ls
```

You should see `linux/arm64` listed among the supported platforms. That single `binfmt` step is what makes everything else work: it teaches the Linux kernel to hand `aarch64` executables to QEMU transparently.

5.4 Build and run the Pi target

The repository ships `deploy/Dockerfile.pi4` and `deploy/docker-compose.pi4.yml`.

PowerShell — from the repo root

```
docker buildx build --platform linux/arm64 `
  -f deploy/Dockerfile.pi4 -t conclave:pi4 --load .

# sanity check: what architecture are we actually on?
docker run --rm --platform linux/arm64 conclave:pi4

# the real check, with a Pi 4's memory ceiling enforced
docker compose -f deploy/docker-compose.pi4.yml run --rm pi4 `
  python deploy/pi_check.py
```

Three details in that Dockerfile are deliberate and worth copying if you write your own:

Line	Why
`--only-binary=:all:`	Makes a missing ARM wheel fail *loudly* in seconds instead of silently starting an hour-long source build. This is the single most useful flag in the file.
`apt-get install libgomp1`	LightGBM's OpenMP runtime. The `python:slim` images do not ship it, and the resulting `ImportError` is cryptic.
`OMP_NUM_THREADS=1`	Matches the single-decision workload, for the reason in §4. The compose file raises it to 4 for the Pi 4 profile.

5.5 The compose profiles

Three boards, with **real** cgroup limits — unlike the `RLIMIT_AS` path in `exp4_edge.py`, which silently does nothing on Windows:

Service	Board	RAM	CPUs	Notes
`pi4`	Pi 4 Model B	4 GB	4	the usual target
`pi3`	Pi 3B+	1 GB	4	tight but workable
`zero2w`	Pi Zero 2 W	512 MB	1	the hard case

equal to **disables swap**

Without that, Docker lets the container swap and your "512 MB" test quietly becomes a 1 GB test. Setting them equal is what makes the ceiling real. This is the most commonly botched part of container-based memory testing.

5.6 What `pi_check.py` reports, and why it is split

`deploy/pi_check.py` trains a small agent, exercises it, and labels every line `[valid]` or invalid, detecting emulation by looking for a real Pi signature in `/proc/cpuinfo` and `/proc/device-tree/model`:

One captured run on native 2-core x86 — your numbers will differ

```
  architecture   : x86_64
  visible cores  : 2
  RAM ceiling    : 3915 MB
  emulated       : no
======================================================================
  [valid]  agent artefact      :    186.2 KB
  [valid]  training accuracy   :     98.19 %
  [valid]  peak RSS            :    194.9 MB
  [valid]  fits RAM ceiling    : YES
  [valid]  training time       :       2.0 s

  [valid (native)]
           median decision     :      2.41 ms
           p99 decision        :      2.92 ms
```

Run the same script under the `arm64` container and the header flips to `emulated: YES`, the timing block is marked invalid, and the script tells you so in plain language.

Latency is noisy; do not quote a single run

Repeated runs of the same benchmark on the same machine vary by tens of percent, depending on thermal state and what else is running. Across three runs of `exp4_edge.py` on the same container the host median moved between 2.4 and 2.8 ms. If a latency number is going into a paper, report the median of several independent runs and say how many — and re-run every experiment that feeds a table before you submit, because a stale table is the easiest mistake to make and the hardest to spot.

5.7 What you may and may not claim

Measurement	Status under emulation
Model / artefact size	**Valid.** Architecture-independent bytes.
Peak RSS	**Valid.** The cgroup limit is real, and pointer width is genuinely 64-bit ARM.
Fits in RAM (yes/no)	**Valid**, and this is the claim that most often decides a deployment.
Correctness / accuracy	**Valid.** Same wheels, same code paths.
Wheel availability	**Valid**, and only obtainable this way.
Decision latency	**Invalid.** QEMU is typically 5–20× slower and not uniformly so.
Energy per decision	**Invalid.** Derived from latency.
Throughput	**Invalid.**

How to phrase this in the paper

The paper currently reports estimated on-device latency scaled by a relative single-core throughput ratio, and labels it an estimate. That is defensible. What is *not* defensible is presenting a QEMU number as a Pi measurement. If a reviewer asks for hardware numbers, the honest answer is: "memory and correctness were verified on an emulated `aarch64` target with enforced cgroup limits; latency is a stated scaling model, and we have not measured on silicon." Buying a Pi 4 is the cheapest way to close that gap.

5.8 When you do get real hardware

On the Pi, over SSH — 64-bit Raspberry Pi OS

```
uname -m                          # want: aarch64
sudo apt update && sudo apt install -y python3-venv python3-dev libgomp1

python3 -m venv ~/conclave-env
source ~/conclave-env/bin/activate

git clone https://github.com/<user>/conclave && cd conclave
pip install --only-binary=:all: -e ".[boost]"

python deploy/pi_check.py         # now every line is valid
pytest -q                         # 9 smoke tests; paper-claim tests need results/
```

Two practical notes for real hardware:

- **Thermal throttling is real.** A Pi 4 under sustained load drops clock speed. Run `vcgencmd measure_temp` alongside your benchmark and discard runs above about 80 °C, or report that you used a heatsink.
- **Pin the governor** before timing anything: `sudo cpufreq-set -g performance`. Otherwise the first few decisions are measured at idle clock and your p99 is meaningless.

5.9 Deploying a trained agent, not the training code

For a field device you ship the artefact, not the pipeline:

On your laptop

```
import joblib
joblib.dump({"fz": fz, "brain": brain, "gate": brain.gate},
            "agent.joblib", compress=3)      # ~200 KB
```

On the Pi — no pandas, no matplotlib, no training data

```
import joblib
from conclave import Agent
from conclave.core.types import Observation

b = joblib.load("agent.joblib")
agent = Agent(brain=b["brain"], featurizer=b["fz"],
              registry=registry, gate=b["gate"])

while True:
    obs = Observation(features=read_sensors(), text=read_radio())
    d = agent.decide(obs)
    log(d.action, d.confidence, d.disagreement, d.gate)
```

The runtime dependency set collapses to `numpy`, `scipy`, `scikit-learn`, `joblib` — and `lightgbm` only if you use `ToolRouter`. That is roughly 60 MB of wheels and a 200 KB agent.

6 The mathematics: where every equation comes from

The paper contains one displayed equation and a dozen implicit ones. None of them are new — that is the point. CONCLAVE's contribution is the *composition*, so every piece should be traceable to established work. This section states each one, says where it comes from, shows where it lives in the code, and explains what it does operationally.

Equation	Origin	In the code
Shannon entropy	Shannon 1948	`brain/uncertainty.py`
Soft-voting aggregation	Bayesian model averaging	`brain/ensemble.py`
Aleatoric/epistemic split	Houlsby et al. 2011 (BALD)	`brain/uncertainty.py`
Margin	Scheffer et al. 2001	`brain/uncertainty.py`
Vote entropy	Dagan & Engelson 1995	`brain/uncertainty.py`
Risk–coverage, AURC	El-Yaniv & Wiener 2010	`brain/uncertainty.py`
TF–IDF	Spärck Jones 1972	`brain/featurize.py`
Hashing trick	Weinberger et al. 2009	`brain/featurize.py`
Kernel-weighted prior	Nadaraya–Watson	`memory/episodic.py`
Linear opinion pool	Stone 1961	`core/agent.py`
A* with admissible h	Hart et al. 1968	`planning/planner.py`
Paired bootstrap	Efron 1979	`experiments/exp3_sequential.py`

6.1 Entropy, and why it is normalised

Shannon entropy of a distribution p over K actions:

$$\mathcal{H}[p] \;=\; -\sum_{k=1}^{K} p_k \log p_k \quad \text{[Shannon 1948]} \tag{1}$$

measured in nats. It is 0 when p puts all mass on one action, and maximal when p is uniform, where $\mathcal{H}[\text{uniform}] = \log K$.

Because CONCLAVE compares uncertainty across problems with different numbers of actions — five tools here, ten humanitarian categories there — every quantity is divided by $\log K$:

$$\tilde{\mathcal{H}}[p] \;=\; \frac{\mathcal{H}[p]}{\log K} \;\in\; [0, 1]. \tag{2}$$

Without this the same gate threshold would mean different things in different deployments. In `conclave/brain/uncertainty.py` this is the `logk` divisor appearing in every return value.

6.2 Aggregating the committee

With M members producing posteriors $p_1, \ldots, p_M$ and weights w_m summing to one, the ensemble posterior is the weighted mean:

$$\bar{p} \;=\; \sum_{m=1}^{M} w_m \, p_m. \tag{3}$$

This is the empirical form of Bayesian model averaging: treat the members as samples from a posterior over models and marginalise. With `weighting="uniform"` all $w_m = 1/M$, which is what the paper uses and is hard to beat when members are comparable in quality.

Why averaging *probabilities* and not votes

Averaging hard votes throws away exactly the information the gate needs. Two committees can both vote 3–1 for the same action while one is unanimous in spirit $(0.9, 0.9, 0.9, 0.4)$ and the other barely decided $(0.51, 0.52, 0.53, 0.49)$. Equation (3) keeps that difference; majority voting destroys it.

6.3 The decomposition — the one equation in the paper

This is the equation that earns the committee its place. Total predictive uncertainty splits into two parts with different operational meanings:

$$\underbrace{\mathcal{H}[\bar{p}]}_{\text{total}} \;=\; \underbrace{\frac{1}{M}\sum_{m=1}^{M} \mathcal{H}[p_m]}_{\textbf{aleatoric}} \;+\; \underbrace{\left(\mathcal{H}[\bar{p}] - \frac{1}{M}\sum_{m=1}^{M} \mathcal{H}[p_m]\right)}_{\textbf{epistemic}} \tag{4}$$

6.3.1 Where it comes from

The right-hand bracket is not an arbitrary residual — it is the **mutual information** between the prediction y and the model identity θ:

$$\mathcal{I}(y; \theta \mid x) = \underbrace{\mathcal{H}\big[\mathbb{E}_\theta \, p(y \mid x, \theta)\big]}_{\mathcal{H}[\bar{p}]} - \underbrace{\mathbb{E}_\theta\big[\mathcal{H}[p(y \mid x, \theta)]\big]}_{\frac{1}{M}\sum_m \mathcal{H}[p_m]} \tag{5}$$

using the standard identity $\mathcal{I}(y; \theta) = \mathcal{H}(y) - \mathcal{H}(y \mid \theta)$ and approximating the expectation over θ by the finite committee. This quantity is the acquisition function from **Bayesian Active Learning**

by Disagreement (BALD), introduced by Houlsby, Huszár, Ghahramani and Lengyel in 2011 for choosing which point to label next. CONCLAVE repurposes it: instead of asking *which example should I label*, it asks *should I act at all*.

Using a deliberately diverse ensemble as the stand-in for a posterior over models follows Lakshminarayanan et al. (2017), who showed ensembles give uncertainty estimates competitive with far more expensive Bayesian methods.

6.3.2 Why the epistemic term cannot go negative

Entropy is concave. By Jensen's inequality, for any weights w_m,

$$\mathcal{H}\Big[\sum_m w_m p_m\Big] \;\geq\; \sum_m w_m\, \mathcal{H}[p_m], \tag{6}$$

so $\mathcal{I} \geq 0$ always, with equality exactly when all members agree. The code clamps at zero anyway (`max(H_mean - mean_H, 0.0)`) purely to absorb floating-point noise.

Verified numerically

Over 4,000 randomly generated committees (2–6 members, 5 actions), the identity in (4) held to 4.4×10^{-16} and the epistemic term was non-negative in every single case. The check is reproducible; the degenerate cases below are the ones worth memorising.

6.3.3 What the two terms actually mean

Committee state	conf	alea	epis	Right response
All agree, confident	0.90	0.36	**0.00**	Act
All agree, all unsure	0.33	**1.00**	0.00	Abstain — more data will not help
Members disagree	0.34	0.33	**0.67**	Gather or escalate — out of competence

Rows two and three are the entire argument for the decomposition. Both have confidence around 0.33. A single model reporting only "33% confident" cannot distinguish them — yet one is a genuinely ambiguous situation where no amount of extra sensing helps, and the other is the model being out of its depth. `UncertaintyGate` routes them to `ABSTAIN` and `GATHER`/`ESCALATE` respectively.

The honest caveat, which is also in the paper

Measured on CONCLAVE's own benchmarks, the decomposition did *not* beat plain maximum-posterior confidence at either selective prediction or shift detection (AUROC 0.955 for the epistemic term versus 0.960 for confidence on novel text). The theory is clean and the degenerate cases above are real, but on these tasks the extra machinery did not pay for itself. Equation (4) is reported because it is what the framework computes, not because the measurement vindicated it.

6.4 Margin and vote entropy

Two cheaper signals the gate also consults. The **margin** is the gap between the top two actions,

$$\text{margin}(\bar{p}) \;=\; \bar{p}_{(1)} - \bar{p}_{(2)}, \text{[Scheffer et al. 2001]} \tag{7}$$

where $\bar{p}_{(i)}$ is the i-th largest component. It catches the case where the top probability is respectable but a rival is right behind it — a situation entropy alone smooths over.

Vote entropy discards the probabilities and looks only at which action each member would pick:

$$V_k = \frac{1}{M}\sum_{m=1}^{M} \mathbf{1}\left[\arg\max_j (p_m)_j = k\right], \qquad \tilde{\mathcal{H}}[V] \text{[Dagan \& Engelson 1995]} \tag{8}$$

This is the original query-by-committee disagreement measure (Seung, Opper and Sompolinsky, 1992). It is robust to a single badly-calibrated member, because one member's overconfidence cannot inflate a hard vote.

6.5 Selective prediction: the risk–coverage curve

An agent that may decline to act is a *selective classifier*: a pair (f, g) where f predicts and $g \in \{0, 1\}$ decides whether to act. Its two figures of merit are

$$\text{coverage} = \mathbb{E}[g(x)], \qquad \text{selective risk} = \frac{\mathbb{E}[\ell(f(x), y)\, g(x)]}{\mathbb{E}[g(x)]}, \text{ [El-Yaniv \& Wiener 2010]} \tag{9}$$

that is, the error rate *among the decisions actually acted upon*. Sweeping the gate threshold traces a curve; the area under it (AURC) summarises how well the uncertainty signal ranks decisions. `risk_coverage_curve` computes it, and CONCLAVE reports AURC $= 0.946$ on the sequential task.

`UncertaintyGate.calibrate` inverts this: given a target precision, it searches a 24×24 grid over (confidence_min, epistemic_max) for the thresholds that **maximise coverage** subject to precision-on-accepted $\geq$ target. That is the standard constrained form of the selective-classification problem, and it is why the gate's thresholds are calibrated rather than guessed.

6.6 Text features: TF–IDF and the hashing trick

6.6.1 TF–IDF, exactly as scikit-learn computes it

For term t in document d, with n documents and $\mathrm{df}(t)$ containing t:

$$\mathrm{tf}'(t, d) = 1 + \log \mathrm{tf}(t, d) \qquad (\texttt{sublinear_tf=True}) \tag{10}$$

$$\mathrm{idf}(t) = \log \frac{1 + n}{1 + \mathrm{df}(t)} + 1 \qquad (\texttt{smooth_idf=True}) \tag{11}$$

$$x_{t,d} = \mathrm{tf}'(t, d) \cdot \mathrm{idf}(t), \qquad x_d \leftarrow x_d / \|x_d\|_2 \qquad \text{[Spärck Jones 1972]} \tag{12}$$

Verified against scikit-learn

Implementing the three lines above by hand reproduces `TfidfVectorizer(sublinear_tf=True)` to 10^{-12}. The $+1$ terms are smoothing, so an unseen term never divides by zero; the sublinear `tf` stops a word repeated twenty times counting twenty times as much.

Because rows are L_2-normalised, the dot product between a query vector and a tool-specification vector *is* their cosine similarity — which is precisely feature #1 of `ToolRouter`.

6.6.2 The hashing trick, and what it costs

Storing a vocabulary makes the model grow with the corpus. Instead, a hash function $h : \text{term} \to \{1, \ldots, D\}$ maps terms directly to indices:

$$\phi_j(x) \;=\; \sum_{i\,:\,h(i)=j} x_i \text{[Weinberger et al. 2009]} \tag{13}$$

The feature space is fixed at D regardless of how much text is seen — there is no stored vocabulary at all. This is why `n_word_features` sets the artefact size directly, and it is the single most important property for microcontroller-class deployment.

CONCLAVE uses the unsigned variant, which is biased

Weinberger et al.'s analysis relies on a *signed* hash, $\xi(i) \in \{\pm 1\}$, which makes collisions cancel in expectation and preserves inner products without bias. CONCLAVE sets `alternate_sign=False`, so colliding terms always add *positively* and similarities are biased slightly upward.

This is a deliberate trade: `ComplementNB` requires non-negative inputs, and keeping a generative member in the committee was judged worth the bias. With $D = 2^{16}$ collisions are rare enough that it does not show up in the measurements, but it is a real approximation and worth knowing if you shrink D aggressively.

6.7 Episodic memory as a kernel-weighted prior

Given a query key z and retrieved cases (z_i, a_i, r_i) with similarity $s_i = \max(0, 1 - d_i)$, the memory produces a prior over actions:

$$\pi_k \;\propto \sum_{i\,:\,a_i=k,\; r_i>0} s_i\, r_i, \qquad \pi \leftarrow \pi^{1/T} \big/ \textstyle\sum_k \pi_k^{1/T} \tag{14}$$

This is a Nadaraya–Watson kernel-weighted average with a truncated-cosine kernel, restricted to cases that were actually rewarded, and T a temperature that sharpens ($T < 1$) or flattens ($T > 1$) the result. When no retrieved case has positive reward it returns the uniform distribution, so memory never biases a decision it knows nothing about.

The result is combined with the ensemble posterior as a **linear opinion pool**:

$$p_{\text{final}} \;=\; (1-\omega)\,\bar{p} \;+\; \omega\,\pi, \qquad \omega = \texttt{memory_weight} = 0.25 \text{[Stone 1961]} \tag{15}$$

A linear pool, unlike a logarithmic one, cannot veto: if either source assigns an action non-negligible mass it survives. For an agent that must not be talked out of a correct action by a stale memory, that is the safer choice.

6.8 The planner, and an honest caveat

`Planner.plan` is A* over sets of predicates. With $g(s)$ the cost so far and $h(s)$ an estimate of the cost remaining, it expands the node minimising $f(s) = g(s) + h(s)$, and A* returns an optimal plan *provided* h never overestimates — the admissibility condition of Hart, Nilsson and Raphael (1968).

CONCLAVE uses the goal-count heuristic:

$$h(s) \;=\; \big|\, G \setminus s \,\big| \tag{16}$$

the number of goal predicates not yet satisfied.

This heuristic is not admissible in general

The docstring calls it admissible. That holds only under two assumptions:

1. every action adds **at most one** goal predicate, and
2. every action costs **at least 1**.

Break either and h can overestimate, and A* may return a suboptimal plan. A concrete case, reproducible in three tools:

```
step_a     : effects {a},        cost 0.1
finish_bc  : requires a, effects {b,c}, cost 0.5   # total 0.6  <- optimal
```

```
direct     : effects {a,b,c},   cost 1.0
# goal = {a,b,c}   ->   CONCLAVE returns ['direct'], cost 1.0
```

At state $\{a\}$ we have $g = 0.1$ and $h = |\{b, c\}| = 2$, so $f = 2.1$ — but the true remaining cost is 0.5. The optimal path is pushed behind the `direct` goal node at $f = 1.0$, which is popped first.

The shipped disaster tools all have single-predicate effects and unit costs, so the guarantee holds there. If you add a multi-effect or sub-unit-cost tool, either accept "good but not provably optimal", or restore admissibility with

$$h(s) \;=\; |G \setminus s| \cdot \frac{c_{\min}}{e_{\max}}, \tag{17}$$

where $c_{\min}$ is the cheapest action cost and $e_{\max}$ the most goal predicates any single action can add. Setting $h \equiv 0$ turns A* into uniform-cost search: always optimal, just slower.

6.9 The paired bootstrap in the headline experiment

The connectivity result compares two policies on *the same* episode seeds, so the comparison is paired. For per-episode differences $d_i = R_i^{\text{CONCLAVE}} - R_i^{\text{cloud}}$, resample indices with replacement $B = 4000$ times and take the empirical 2.5th and 97.5th percentiles of $\bar{d}^*$:

$$\bar{d}^{*(b)} = \frac{1}{n}\sum_{i=1}^{n} d_{\,\iota_i^{(b)}}, \qquad \mathrm{CI}_{95} = \Big[\, Q_{2.5}(\bar{d}^*),\; Q_{97.5}(\bar{d}^*) \Big] \text{ [Efron 1979]} \tag{18}$$

Pairing matters here: episode difficulty varies enormously (when the surge arrives, how big it is), and pairing removes that variance instead of letting it swamp the effect. It is also what lets the paper make its most important negative claim — that at zero outage the two are *indistinguishable*, $\Delta = 0.6$ with CI $[-0.7, 1.9]$ — which is the check that the comparison was fair in the first place.

6.10 Reading list, in dependency order

1. **Shannon (1948)**, *A Mathematical Theory of Communication* — entropy.
2. **Spärck Jones (1972)** — inverse document frequency.
3. **Hart, Nilsson & Raphael (1968)** — A* and admissibility.
4. **Seung, Opper & Sompolinsky (1992)**; **Dagan & Engelson (1995)** — query by committee, vote entropy.
5. **Breiman (2001)** — random forests; where ensemble diversity comes from.
6. **Rennie et al. (2003)** — complement naive Bayes.
7. **Weinberger et al. (2009)** — the hashing trick and its guarantees.
8. **El-Yaniv & Wiener (2010)** — selective classification.
9. **Houlsby et al. (2011)** — BALD; equation (4). *Start here if you read only one.*
10. **Lakshminarayanan et al. (2017)** — ensembles as practical uncertainty.

Appendix — one-page reference

Commands

Setup, test, explore

```
py -3.12 -m venv .venv                          # create
.\.venv\Scripts\Activate.ps1                    # activate (PowerShell)
pip install -e ".[boost,experiments,dev]"       # install

pytest -q                                       # expect 14 passed
python demo.py episode                          # watch the net column
python demo.py uncertainty                      # break the sensors
python examples/pump_station.py                 # the worked example

python scripts/run_all.py                       # reproduce the paper
python experiments/make_figures.py              # redraw figures
```

Emulated Raspberry Pi

```
docker run --privileged --rm tonistiigi/binfmt --install arm64
docker buildx build --platform linux/arm64 -f deploy/Dockerfile.pi4 -t conclave:pi4
    --load .
docker compose -f deploy/docker-compose.pi4.yml run --rm pi4 python deploy/pi_check
    .py
```

The six-step program

Minimal complete agent

```
from conclave import (Agent, EnsembleBrain, EpisodicMemory, Observation,
                      ToolRegistry, UncertaintyGate)
from conclave.brain.featurize import (BudgetedTextFeaturizer, FusedFeaturizer,
                                      TabularFfeaturizer)
from conclave.tools.base import tool

@tool(description="What a user would say they need")          # 1. tools
def do_something(arg: int = 1) -> str: ...
registry = ToolRegistry([do_something])

observations, labels = load_my_decision_log()                 # 2. data

fz = FusedFeaturizer(                                         # 3. features
    text=BudgetedTextFeaturizer(n_word_features=2**14,
                                n_char_features=2**14),
    tabular=TabularFeaturizer()).fit(observations)

brain = EnsembleBrain(weighting="logloss").fit(               # 4. brain
    fz.transform(observations), labels)

gate = brain.calibrate_gate(X_val, y_val, target_precision=0.90)    # 5. gate

agent = Agent(brain=brain, featurizer=fz, registry=registry,  # 6. agent
              gate=gate, memory=EpisodicMemory(capacity=2000))

d = agent.decide(Observation(features={...}, text="..."))
print(d.action, d.confidence, d.disagreement, d.gate)
```

Uncertainty cheat-sheet

Aleatoric	Epistemic	Meaning	Gate fires
low	low	Clear-cut	`ACT`
high	low	Genuinely ambiguous; more data will not help	`ABSTAIN`
low	high	Members disagree; out of competence	`GATHER` / `ESCALATE`
high	high	Hard *and* unfamiliar	`ESCALATE`

Numbers worth remembering

Quantity	Value
Tool routing, BFCL `live_multiple`	90.8% top-1, 4.0 ms, 631 KB
Routing on tools unseen in training	78.9% (lexical baseline 56.7%)
Abstention (irrelevance detection)	84.3% accuracy, 0.921 AUC
HumAID triage, 2^{16} features	0.743 weighted F1 vs RoBERTa 0.760
Disaster agent artefact	232 KB
Decision latency (2-core x86)	$\approx$ 2.5–2.8 ms median
Return at 73% offline	CONCLAVE +33.5 vs cloud oracle −127.3
Selective prediction	96.3% at full coverage $\rightarrow$ 100% at 50%
Shift detection, novel text	AUROC 0.955

Things that are true and inconvenient

1. Text features are lexical. Paraphrase and negation hurt more than they hurt a transformer.
2. The aleatoric/epistemic split did not beat plain confidence on these benchmarks.
3. Uncertainty-triggered human oversight lost to *random* oversight in the sequential task.
4. Peak RSS is $\approx$ 200 MB, dominated by the Python runtime; the agent itself is 232 KB.
5. The planner heuristic is not admissible for multi-effect or sub-unit-cost tools.
6. On-device latency is a scaling model, not a measurement. No Raspberry Pi was harmed in the making of this paper — because none was used.